

Predicting Online Ad Click-Through

with Logistic Regression

Machine Learning Practice With Python

Siksha 'O' Anushandhan, ITER

Center for Data Science

Table of Contents

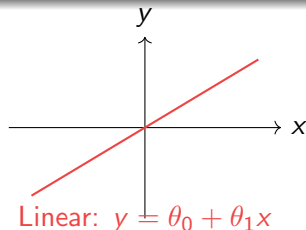
- 1 The Problem: From Linear to Logistic
- 2 Mathematical Foundation of Logistic Regression
- 3 Implement “Predicting Online Ad Click-Through with Logistic Regression”
- 4 Implement Online Learning method



The Problem: From Linear to Probabilistic

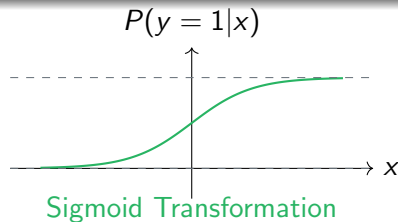
⚠ Linear Regression Issue

- **Output Range:** $(-\infty, +\infty)$
- **Problem:** Unbounded values
- **Issue:** Can't represent probabilities
- **Example:** $\hat{y} = -2.5$ or $\hat{y} = 3.7$



💡 What We Need

- **Output Range:** $(0, 1)$
- **Goal:** Probability values
- **Interpretation:** $P(y = 1|x)$
- **Example:** $P = 0.85$ (85% chance)



➔ **Solution:** Sigmoid Function transforms any real number to $[0,1]$



The Sigmoid Function: Mathematical Elegance

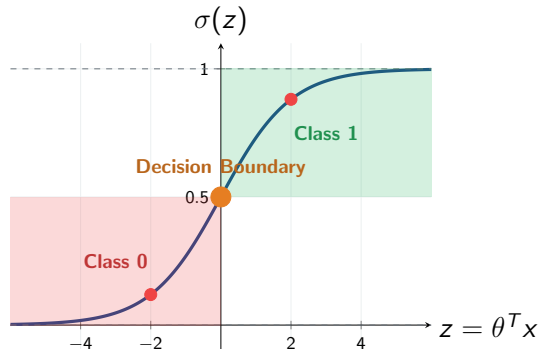
⚙ The Sigmoid Formula

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

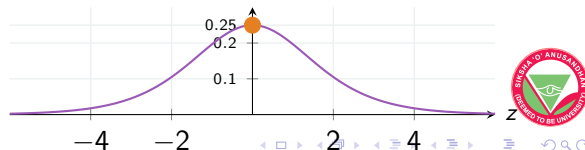
where $z = \theta^T x = \theta_0 + \theta_1 x_1 + \dots + \theta_n x_n$

📊 Key Properties

- **Domain:** $(-\infty, +\infty)$
- **Range:** $(0, 1)$
- **Shape:** S-shaped curve
- **Center:** $\sigma(0) = 0.5$
- **Monotonic:** Always increasing



Sigmoid Derivative: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$



Real-World Applications: Where Sigmoid Shines



Classification Tasks

-  **Email:** Features $\rightarrow P(\text{spam})$
-  **Medical:** Symptoms $\rightarrow P(\text{disease})$
-  **E-commerce:** Behavior $\rightarrow P(\text{buy})$



Decision & Comparison

$\sigma(Z) \leq 0.5$	Class 0
$\sigma(Z) > 0.5$	Class 1

0.5 threshold

Function	Range	Smooth
Sigmoid	$(0, 1)$	✓
Step	$\{0, 1\}$	✗
Linear	$(-\infty, \infty)$	✓



Neural Networks

- **Activation Function:** Historical importance
- **Output Layer:** Still widely used for binary classification
- **Gating Mechanisms:** LSTM, GRU cells
- **Attention Mechanisms:** Soft attention weights

Key Insight: Sigmoid transforms any real input into interpretable probabilities $(0, 1)$ while maintaining differentiability for optimization.

Mathematical Foundation of Logistic Regression

Model Definition

Hypothesis Function:

$$h_{\theta}(x) = \sigma(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

Probabilistic Interpretation:

$$P(y = 1|x; \theta) = h_{\theta}(x)$$

$$P(y = 0|x; \theta) = 1 - h_{\theta}(x)$$

Compact Form:

$$P(y|x; \theta) = (h_{\theta}(x))^y (1 - h_{\theta}(x))^{1-y}$$

Likelihood for m samples

$$L(\theta) = \prod_{i=1}^m (h_{\theta}(x^{(i)}))^{y^{(i)}} (1 - h_{\theta}(x^{(i)}))^{1-y^{(i)}}$$

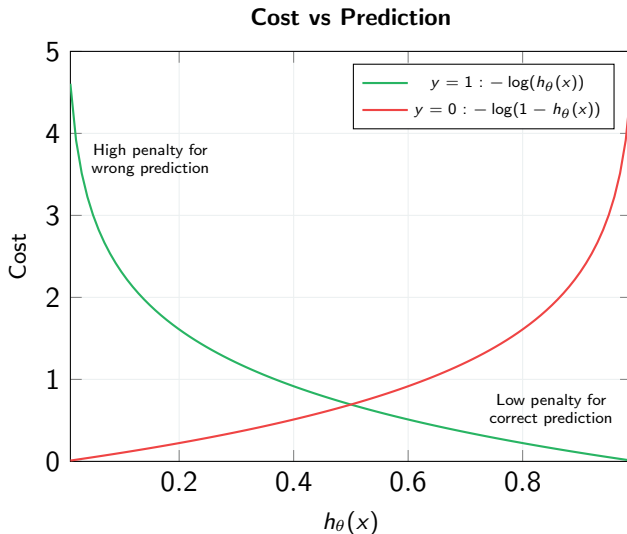
Log-Likelihood

$$\begin{aligned} \mathcal{L}(\theta) = \log(L(\theta)) &= \sum_{i=1}^m \left[y^{(i)} \log(h_{\theta}(x^{(i)})) \right. \\ &\quad \left. + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right] \end{aligned}$$

Cost function

$$\begin{aligned} J(\theta) &= -\frac{1}{m} \mathcal{L}(\theta) \\ &= -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(h_{\theta}(x^{(i)})) \right. \\ &\quad \left. + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right] \end{aligned}$$

Cost function of Logistic Regression



$J(\theta)$ is convex, therefore, it has global minimum. Gradient descent method ensure to reach global minimum point.



Optimize parameters of cost function

Optimize θ by gradient descent

- Initialize θ
- Update θ

$$\theta := \theta - \eta \frac{\partial J(\theta)}{\partial \theta}$$

$$\frac{\partial J(\theta)}{\partial \theta} = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} - h_{\theta}(x^{(i)})] x^{(i)} = \frac{1}{m} X^T [h_{\theta}(X) - Y]$$

- Update until it reach termination point.



Optimize parameters of cost function

Cost function with penalty

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right] + \alpha (\|\theta\|_p)^p$$

Gradient for L2 regularization

Cost function

$$J_{reg}(\theta) = J(\theta) + \frac{\alpha}{2m} (\|\theta\|_2)^2$$

Gradient

$$\frac{\partial J_{reg}(\theta)}{\partial \theta} = \frac{1}{m} X^T (h_{\theta}(X) - Y) + \frac{\alpha}{m} \begin{bmatrix} 0 \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix}$$

Optimize parameters of cost function

Exercise:

Suppose you know that a model is overfitted (without regularization term) and you know that the reason of overfitting is huge number of features. To reduce the overfitting, which regularization term you will use?



Implement “Predicting Online Ad Click-Through with Logistic Regression”

- Consider the following `x_train` and `y_train`

```
1 x_train = np.array([[6,7], [2,4], [3,6], [4,7], [1,6], [5,2],  
                      [2,0], [6,3], [4,1], [7,2]])  
2 y_train = np.array([0, 0, 0, 0, 0, 1, 1, 1, 1, 1])
```

- Create Logistic regression model, train it, and print coefficient, intercept



Implement “Predicting Online Ad Click-Through with Logistic Regression”

- Consider the following `x_train` and `y_train`

```
1 x_train = np.array([[6,7], [2,4], [3,6], [4,7], [1,6], [5,2],  
                      [2,0], [6,3], [4,1], [7,2]])  
2 y_train = np.array([0, 0, 0, 0, 0, 1, 1, 1, 1, 1])
```

- Create Logistic regression model, train it, and print coefficient, intercept

```
1 from sklearn.linear_model import SGDClassifier  
2 model = SGDClassifier(loss='log_loss', max_iter=1000,  
                       alpha=0.01, penalty='l2')  
3 model.fit(x_train, y_train)  
4  
5 print("Coefficients:", model.coef_)  
6 print("Intercept:", model.intercept_)
```

```
Coefficients: [[ 9.30299536 -9.48918847]]  
Intercept: [0.6378134]
```



Implement “Predicting Online Ad Click-Through with Logistic Regression”

- Predict class for test data for $[[6,1], [1,3], [3,1], [4,5]]$



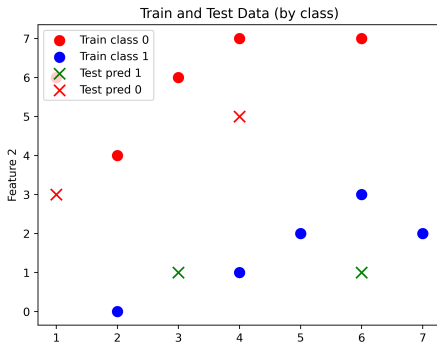
Implement “Predicting Online Ad Click-Through with Logistic Regression”

- Predict class for test data for `[[6,1], [1,3], [3,1], [4,5]]`

```
1 x_test = np.array([[6,1], [1,3], [3,1], [4,5]])
2 y_pred = model.predict(x_test)
3 y_pred
```

```
array([1, 0, 1, 0])
```

- Plot train and test data



Implement “Predicting Online Ad Click-Through with Logistic Regression”

- Load `train.csv`, separate into `X` and `Y`, do necessary encoding, split into train and test data [as previous]



Implement “Predicting Online Ad Click-Through with Logistic Regression”

- Load `train.csv`, separate into `X` and `Y`, do necessary encoding, split into train and test data [as previous]
- Create and train Logistic regression model, predict for test data



Implement “Predicting Online Ad Click-Through with Logistic Regression”

- Load `train.csv`, separate into X and Y, do necessary encoding, split into train and test data [as previous]
- Create and train Logistic regression model, predict for test data

```
1 from sklearn.linear_model import SGDClassifier
2 model = SGDClassifier(loss='log_loss', max_iter=100,
3                       alpha=0.01, penalty='l2')
4 model.fit(X_train, Y_train.values.flatten())
5 y_pred = model.predict(X_test)
```

- Calculate accuracy and `roc_auc_score`



Implement “Predicting Online Ad Click-Through with Logistic Regression”

- Load train.csv, separate into X and Y, do necessary encoding, split into train and test data [as previous]
- Create and train Logistic regression model, predict for test data

```
1 from sklearn.linear_model import SGDClassifier
2 model = SGDClassifier(loss='log_loss', max_iter=100,
3                       alpha=0.01, penalty='l2')
4 model.fit(X_train, Y_train.values.flatten())
5 y_pred = model.predict(X_test)
```

- Calculate accuracy and roc_auc_score

```
1 from sklearn.metrics import accuracy_score, roc_auc_score
2 print("Accuracy:", accuracy_score(Y_test, y_pred))
3 print("ROC AUC:", roc_auc_score(Y_test, y_pred))
```



Implement “Predicting Online Ad Click-Through with Logistic Regression”

- Get all features name.



Implement “Predicting Online Ad Click-Through with Logistic Regression”

- Get all **features name**.

```
1 feature_names = enc.get_feature_names_out()
```

- Print 10 **least important** features.



Implement “Predicting Online Ad Click-Through with Logistic Regression”

- Get all **features name**.

```
1 feature_names = enc.get_feature_names_out()
```

- Print 10 **least important** features.

```
1 print(feature_names[np.argsort(abs(model.coef_))[0][:10]])
```

- Print top 10 **most important** features.



Implement “Predicting Online Ad Click-Through with Logistic Regression”

- Get all **features name**.

```
1 feature_names = enc.get_feature_names_out()
```

- Print 10 **least important** features.

```
1 print(feature_names[np.argsort(abs(model.coef_))[0][:10]])
```

- Print top 10 **most important** features.

```
1 print(feature_names[np.argsort(abs(model.coef_))[0][-10:]])
```



Implement “Predicting Online Ad Click-Through with Logistic Regression”

- Get all **features name**.

```
1 feature_names = enc.get_feature_names_out()
```

- Print 10 **least important** features.

```
1 print(feature_names[np.argsort(abs(model.coef_))[0][:10]])
```

- Print top 10 **most important** features.

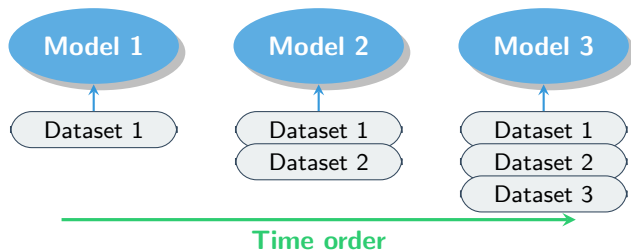
```
1 print(feature_names[np.argsort(abs(model.coef_))[0][-10:]])
```

Exercise:

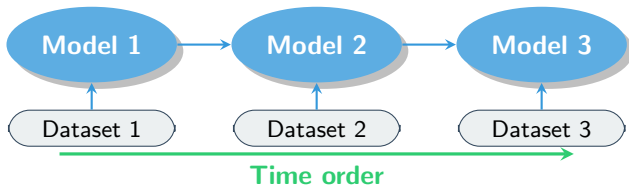
Define Random Forest model for classification, train the model by above dataset and find accuracy, AUC_ROC score. Find 10 most important and 10 least important features for this model.

Learning Paradigms: Online vs Offline Learning

Offline Learning



Online Learning



Key Difference: Online learning processes data sequentially as it arrives, while offline learning uses cumulative datasets with each successive model having access to all previous data.

Train model by online learning method

Train model sequentially with batches by using `partial_fit` function.

```
1 from sklearn.linear_model import SGDClassifier
2 from sklearn.metrics import accuracy_score, roc_auc_score
3 online_model = SGDClassifier(loss='log_loss', max_iter=1000,
4                               learning_rate='constant', eta0 = 0.01, penalty='l2')
5 classes = np.unique(Y_train.values.flatten())
6 batch_size = 1000
7 for i in range(0, X_train.shape[0], batch_size):
8     X_batch = X_train[i:i+batch_size]
9     Y_batch = Y_train.values.flatten()[i:i+batch_size]
10    online_model.partial_fit(X_batch, Y_batch, classes=classes)
11 y_online_pred = online_model.predict(X_test)
12 print("Online Learning Accuracy:", accuracy_score(Y_test,
13                                                    y_online_pred))
14 print("Online Learning ROC AUC:", roc_auc_score(Y_test,
15                                                  y_online_pred))
```

Logistic regression for multi-class

Given

- **Input vector:** $x \in \mathbb{R}^n$
- **Parameters:** Weight matrix $\Theta \in \mathbb{R}^{n \times k}$, where k = number of classes
- **Class labels:** $y \in \{1, 2, \dots, k\}$

Goal

Learn parameters Θ to predict class probabilities:

$$P(y = j \mid x; \Theta) \text{ for all classes } j = 1, 2, \dots, k$$

Key Insight: Unlike binary classification, we need a probability distribution over multiple classes



Hypothesis/prediction Function (Softmax)

Compute Scores (Logits)

$$z_j = \theta_j^\top x \quad \text{for } j = 1, 2, \dots, k$$

where θ_j is the parameter vector for class j

Prediction function (Softmax)

$$h_{\Theta}(x)_j = P(y = j \mid x; \Theta) = \frac{e^{z_j}}{\sum_{\ell=1}^k e^{z_{\ell}}}$$

Properties of Softmax

- $h_{\Theta}(x)_j \in (0, 1)$ for all j
- $\sum_{j=1}^k h_{\Theta}(x)_j = 1$ (valid probability distribution)
- Amplifies differences between scores (exponential function)

Cost Function Development

Maximum Likelihood Approach

We want to **maximize** the probability of correct predictions:

$$\text{Likelihood} = \prod_{i=1}^m P(y^{(i)} \mid x^{(i)}; \Theta)$$

Log-Likelihood

$$\ell(\Theta) = \sum_{i=1}^m \log(h_{\Theta}(x^{(i)})_{y^{(i)}})$$

Cost Function (Negative Log-Likelihood)

$$\begin{aligned} J(\Theta) &= -\frac{1}{m} \sum_{i=1}^m \log(h_{\Theta}(x^{(i)})_{y^{(i)}}) \\ &= -\frac{1}{m} \sum_{i=1}^m \sum_{j=1}^k y_j^{(i)} \log(h_{\Theta}(x^{(i)})_j) \end{aligned}$$



Matrix Form & Gradient

Matrix Notation

- $X \in \mathbb{R}^{m \times n}$: Input matrix
- $H \in \mathbb{R}^{m \times k}$: Predicted probabilities
- $Y \in \mathbb{R}^{m \times k}$: One-hot true labels

Cost becomes:

$$J(\Theta) = -\frac{1}{m} \text{tr}(Y^\top \log H)$$

Gradient for Optimization

$$\nabla_{\Theta} J(\Theta) = \frac{1}{m} X^\top (H - Y)$$

Intuition: Gradient points in direction of steepest increase of cost

Beautiful Property

The gradient has the same form as linear regression! $(H - Y)$ represents the error between predictions and true labels.

Logistic Regression for multi-class classification

- Generate 1000 samples with 3 class and 20 features using `make_classification` module.

```
1 from sklearn.datasets import make_classification
2 X, y = make_classification(n_samples=1000, n_features=20,
    n_informative=15, n_redundant=5, n_classes=3,
    random_state=42)
3 print(X.shape, y.shape)
```

- `n_informative`: The number of informative features for each class.
- `n_redundant`: The number of redundant features. These features are generated as random linear combinations of the informative features.



Logistic Regression for multi-class classification

- Generate 1000 samples with 3 class and 20 features using `make_classification` module.

```
1 from sklearn.datasets import make_classification
2 X, y = make_classification(n_samples=1000, n_features=20,
    n_informative=15, n_redundant=5, n_classes=3,
    random_state=42)
3 print(X.shape, y.shape)
```

- `n_informative`: The number of informative features for each class.
- `n_redundant`: The number of redundant features. These features are generated as random linear combinations of the informative features.

- Define multi-class Logistic regression model

```
1 from sklearn.linear_model import LogisticRegression
2 model = LogisticRegression(multi_class='multinomial',
    solver='lbfgs', max_iter=500)
```

`solver: lbfgs, liblinear, newton-cg, newton-cholesky, sag, saga` (Exercise)



Logistic Regression for multi-class classification

- Split the dataset into train and test, fit the model, and calculate accuracy, f1-score.



Logistic Regression for multi-class classification

- Split the dataset into train and test, fit the model, and calculate accuracy, f1-score.

```
1 from sklearn.model_selection import train_test_split
2 from sklearn.metrics import accuracy_score, f1_score
3 X_train, X_test, y_train, y_test = train_test_split(X, y,
4                                                    test_size=0.2, random_state=42)
5 model.fit(X_train, y_train)
6 y_pred = model.predict(X_test)
7 print("Accuracy:", accuracy_score(y_test, y_pred))
8 print("F1 Score:", f1_score(y_test, y_pred,
9                               average='weighted'))
```



Exercise:1

Logistic regression is often used for binary classification problems such as predicting whether a user will click on an ad. Why do we use the logistic (sigmoid) function instead of a simple linear regression model for such tasks?

Exercise:2

Imagine you trained a logistic regression model for click-through prediction and achieved very high accuracy, but the AUC is low. What could this indicate about your dataset or model?

Exercise:3

Logistic regression outputs probabilities between 0 and 1. If you change the decision threshold from 0.5 to 0.3, what impact would that have on click-through prediction?

Exercise:4

Why does choosing a learning rate that is too large or too small negatively affect the performance of gradient descent? How would you decide on an appropriate learning rate for training a logistic regression model?

Exercise:5

In the logistic regression-based click-through prediction project, can you also tweak hyperparameters such as `penalty`, `eta0`, and `alpha` in the `SGDClassifier` model? What is the highest testing AUC you are able to achieve?

Exercise:6

Suppose your model shows signs of overfitting. How would adding L1 vs. L2 regularization affect the model differently? Which one would you prefer and why?

Exercise:7

Online ad data can be extremely large (millions of samples). Why might stochastic gradient descent (SGD) be more suitable than batch gradient descent in this case?

